Data Engineering Assignment 1: Data Pipeline Design

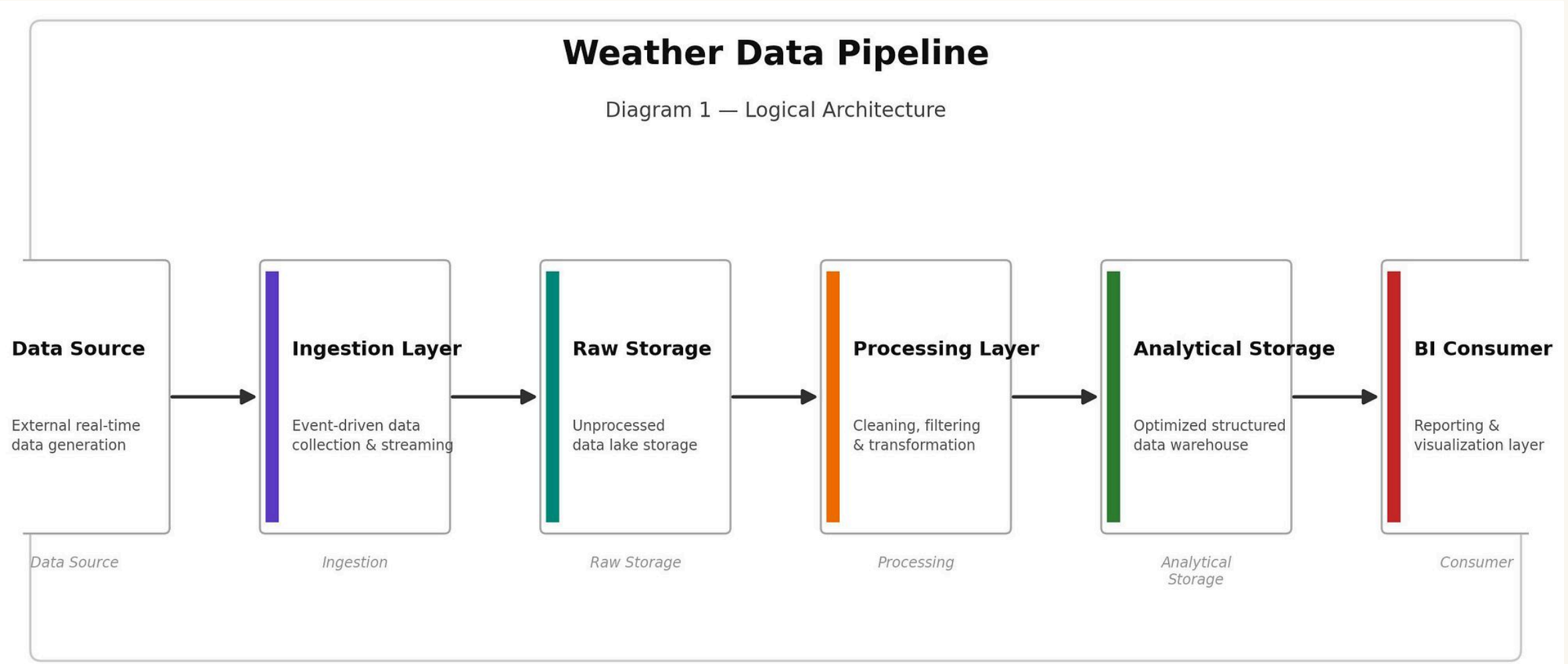
Submitted by :- Soham Jagtap (G1 : DE)

Selected Source

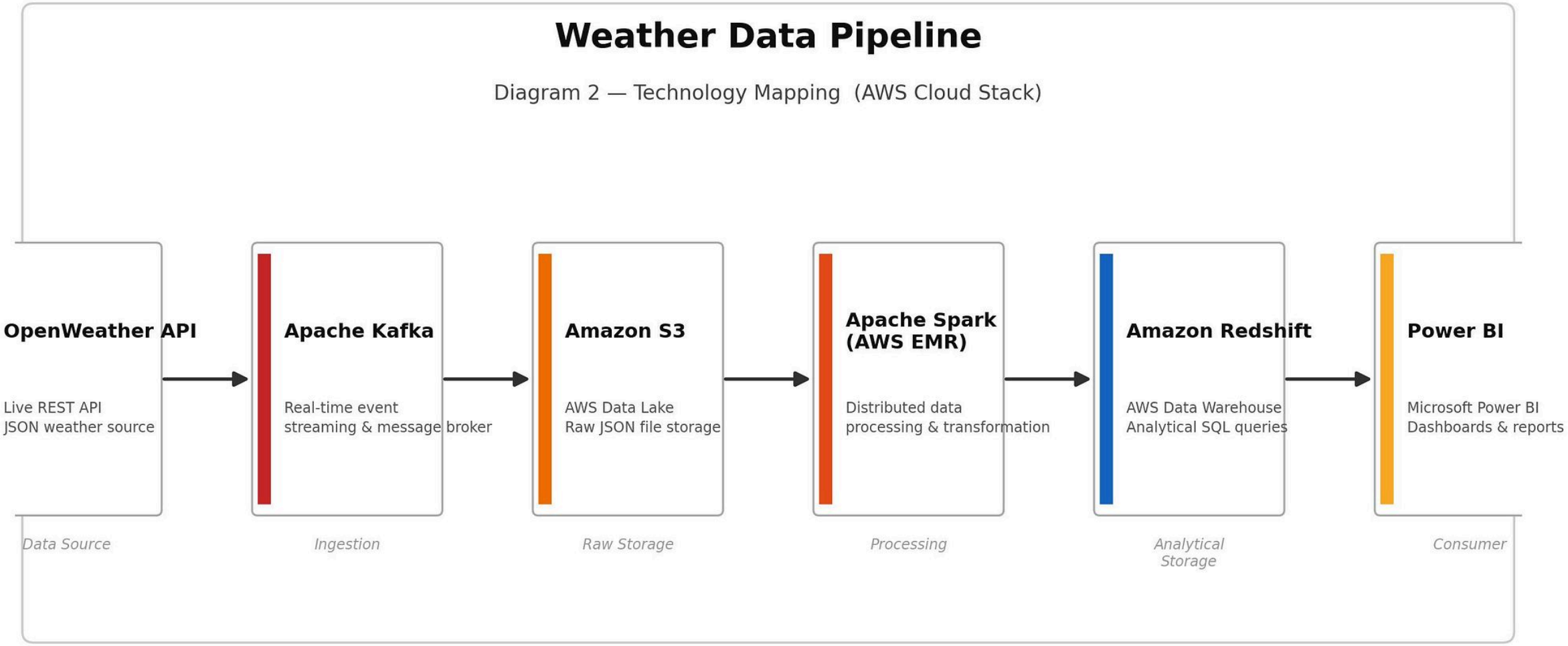
01 **System/Source:** OpenWeather API (Current Weather Data API)

02 **Type of data produced:** JSON format containing nested weather attributes such as temperature, humidity, wind speed, atmospheric pressure, geographical coordinates, and Unix timestamps.

Logical Architecture



Technology Mapping



Tech Stack Used:

01 OpenWeather API

02 Apache Kafka

03 Amazon S3

04 Apache Spark (AWS EMR)

05 Amazon Redshift

06 Power BI

Component Explanation

OpenWeather API: Acts as the real-time source of raw weather data in JSON format.

Apache Kafka: Used to stream the incoming API events reliably into the pipeline as a message queue, ensuring no data is lost even during high traffic.

Amazon S3: Serves as the AWS data lake to store all raw, unprocessed JSON files before any transformation—acts as a safety net for future reprocessing.

Apache Spark (AWS EMR): Used on AWS Elastic MapReduce for distributed processing—cleans, filters, flattens nested JSON, and converts data into a structured tabular format at scale.

Amazon Redshift: An AWS columnar data warehouse used to store the transformed, structured data so it can be queried efficiently using SQL for analytics.

Power BI: Used by business analysts to connect to Amazon Redshift and build interactive dashboards and visual reports that track weather trends over time.

Question Log

01 If the OpenWeather API gives us live data, why can't Power BI just connect directly to the API instead of going through Kafka and Redshift?

02 Why do we need Apache Kafka at all? Can't we just use a Python script to call the API directly every hour?

03 Why is Amazon S3 needed as raw storage if Apache Spark is going to process the data anyway — why not skip S3 and process directly?

04 What happens if the OpenWeather API changes its JSON field names—will our entire Spark processing job on EMR break silently?

05 What is the difference between Amazon S3 and Amazon Redshift—aren't both just storage?

Short Architecture Explanation

Why does data move from one component to the next?

Data originates at the OpenWeather API, which publishes live weather readings in JSON format. It flows into Apache Kafka because Kafka acts as a reliable message broker — it decouples the data source from the rest of the pipeline so that even if a downstream system is slow, no data is dropped. From Kafka, the raw JSON events are landed into Amazon S3 to create a permanent archive of all original data on AWS; this ensures we can always reprocess if something goes wrong later.

Next, Apache Spark running on AWS EMR reads from S3 and performs the heavy lifting — it cleans missing fields, flattens deeply nested JSON into flat columns, and converts timestamps into readable formats. Spark on EMR is used here because it can process large volumes of data in a distributed and cost-effective manner on AWS infrastructure. The clean, flat data is then written into Amazon Redshift, a columnar data warehouse on AWS optimized for fast analytical SQL queries at scale. Finally, Power BI connects directly to Amazon Redshift to let business users build interactive charts, trend graphs, and weather reports — without ever needing to write code themselves.

Design Decisions: What I Chose and Why

Component	Decision	Alternative I Considered	Source I Read	What It Changed in My Design
Ingestion: Apache Kafka	Apache Kafka	Scheduled Python script or AWS Lambda writing straight to S3	S2	Kafka keeps messages for a retention period and lets several consumers read them. I added a producer between the API and Kafka, and I now see Kafka as optional at this scale.
Raw storage: Amazon S3	Amazon S3	Process directly, with no raw layer	S3	S3 is cheap and durable, so I keep the original JSON and can reprocess if the transformation logic changes.
Processing: Spark on EMR	Spark on EMR	AWS Glue (serverless Spark)	S4	Glue needs no cluster management and suits small jobs, while EMR gives more control. I kept EMR and note Glue as the simpler option.
Warehouse: Amazon Redshift	Amazon Redshift	Amazon Athena querying S3 directly	S5	Redshift is a columnar warehouse suited to repeated dashboard queries. Athena is a serverless alternative for occasional queries.
BI: Power BI	Power BI	Amazon QuickSight	S6	I learned that Power BI has a built-in Amazon Redshift connector that supports Import and DirectQuery modes, so analysts can connect with server, database and login details and build reports without writing code.

Answers to My Questions (From Research)

1. **Why not connect Power BI directly to the API?** The API returns only the current reading for one request, with no history. Dashboards need stored history to show trends, and the API has call limits. Storing readings in Redshift lets many users query them without extra API calls. (S1, S5)
2. **Why Kafka instead of an hourly Python script?** For hourly data from a few cities, the script is enough. Kafka helps when there are many sources or consumers, or when a downstream system might be slow, because it buffers and retains messages. (S2)
3. **What is the difference between Amazon S3 and Amazon Redshift?** S3 is object storage for raw files of any format, cheap and durable. Redshift is a columnar SQL warehouse for structured data, built for fast analytical queries. S3 holds the raw record and Redshift serves the analysis. (S3, S5)

Design Corrections After Research

- OpenWeather is a pull-based REST API. A scheduled producer (Python on EventBridge/Lambda) polls it and writes events to Kafka. I added it to the diagrams.
- Kafka retains messages according to its retention and replication settings, so I say “reduces data loss risk”, not “no data lost”.
- Kafka to S3: I use the Kafka Connect S3 sink connector. Amazon Data Firehose was the managed alternative.
- Schema changes: Spark reads with an explicit schema and keeps malformed rows instead of failing. A check counts null rates per column and alerts if a field is suddenly empty. The raw JSON stays in S3, so I can fix the code and reprocess.

Trade-offs and Limitations

- **Cost:** EMR and a Kafka cluster cost more to run than the data volume justifies. S3 storage is very cheap.
- **Scale:** I assume 100 cities polled hourly. That is 2,400 calls a day, about 72,000 a month, under the free-tier limits referenced in my notes. Do not say the limits were verified.
- **Conclusion:** At this size, Lambda → S3 → Glue → Athena or Redshift would be simpler and cheaper. I kept Kafka and EMR to show the streaming-pipeline design, and I'd simplify in a real deployment.
- **What can break:** API rate limits or outages, field name changes, producer failure that leaves gaps in the data, Kafka retention expiring before consumers catch up, and Redshift load failures.

Sources I Actually Used

S1 — OpenWeather: Current Weather Data

https://openweathermap.org/current

I learned that the API returns the current reading on request and doesn't push data, so I needed a producer.

S2 — Apache Kafka Documentation

https://kafka.apache.org/documentation/

I learned that Kafka stores events durably with configurable retention and replication, which is buffering, not a guarantee of zero loss.

S3 — AWS: What is a Data Lake?

https://aws.amazon.com/what-is/data-lake/

I learned that a data lake keeps raw data in its original format, so I can reprocess it later.

S4 — Apache Spark: Spark SQL and DataFrames Guide

https://spark.apache.org/docs/latest/sql-programming-guide.html

I learned that I can define an explicit schema when reading JSON, which helps with field changes.

S5 — Amazon Redshift Database Developer Guide

https://docs.aws.amazon.com/redshift/latest/dg/

I learned that Redshift stores data by column, which suits analytical queries more than raw file storage does.

S6 — Building Power BI Reports on top of Amazon Redshift data (Microsoft Power BI Blog)

https://powerbi.microsoft.com/en-us/blog/building-power-bi-reports-on-top-of-amazon-redshift-data

I learned that Power BI has a built-in Amazon Redshift connector that supports Import and DirectQuery modes, so analysts can connect with server, database and login details and build reports without writing code.